

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

EXPERIMENT 10

Student Name: Aryan Yadav

UID: 24BCF10079

Branch: AIT-CSE (FSD)

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: Computer Network

Subject Code: 24CSH-337

Experiment

Capturing & Analysing Network Packets using Wireshark

Aim

To capture and analyse live network traffic using the Wireshark packet analyzer, and to study the structure and behaviour of TCP/TLS, DNS-over-UDP, QUIC, and ICMPv6 exchanges.

Learning Objectives

1. Understand how a packet analyzer such as Wireshark captures traffic at the network-interface level.
2. Understand promiscuous mode, monitor mode, and the role of libpcap/Npcap.
3. Choose the correct network interface before starting a capture.
4. Capture and analyse an in-progress TCP/TLS session, including a connection teardown.
5. Capture and analyse DNS queries and responses carried over UDP.
6. Capture and analyse QUIC (HTTP/3) traffic and recognise why its payload appears encrypted in Wireshark.
7. Correlate a ping command with the ICMPv6 traffic and DNS lookups it generates.

EXPERIMENT 10

Theory

What is Wireshark Doing Behind the Scenes?

A network interface card (NIC) normally hands a program only the packets addressed to its own device. Wireshark needs to see all traffic on the segment, so it puts the NIC into a special capture mode:

- Promiscuous Mode — on a wired/Wi-Fi connection, the NIC is told to stop discarding packets addressed to other devices, so Wireshark can capture every packet on the network segment, not just its own.
- Monitor Mode (wireless) — on Wi-Fi, monitor mode goes further and captures all wireless frames in the air, even ones not addressed to this device at all.
- libpcap / Npcap — Wireshark doesn't talk to the NIC directly; it relies on a capture library (libpcap on Linux/macOS, Npcap on Windows) to pull raw packets out of the OS kernel.

PC (network traffic) → libpcap/Npcap (fetches from kernel) → NIC (promiscuous/monitor mode) → Wireshark (packet sniffing)

Protocols Captured in This Experiment

This capture was taken on an active Wi-Fi interface carrying normal day-to-day traffic, so alongside the protocols named in the objectives above, it also shows TLSv1.2 Application Data, DNS, QUIC, and plain TCP ACKs — a realistic mix of what a modern device sends and receives even for a single web/app session.

Network Interfaces

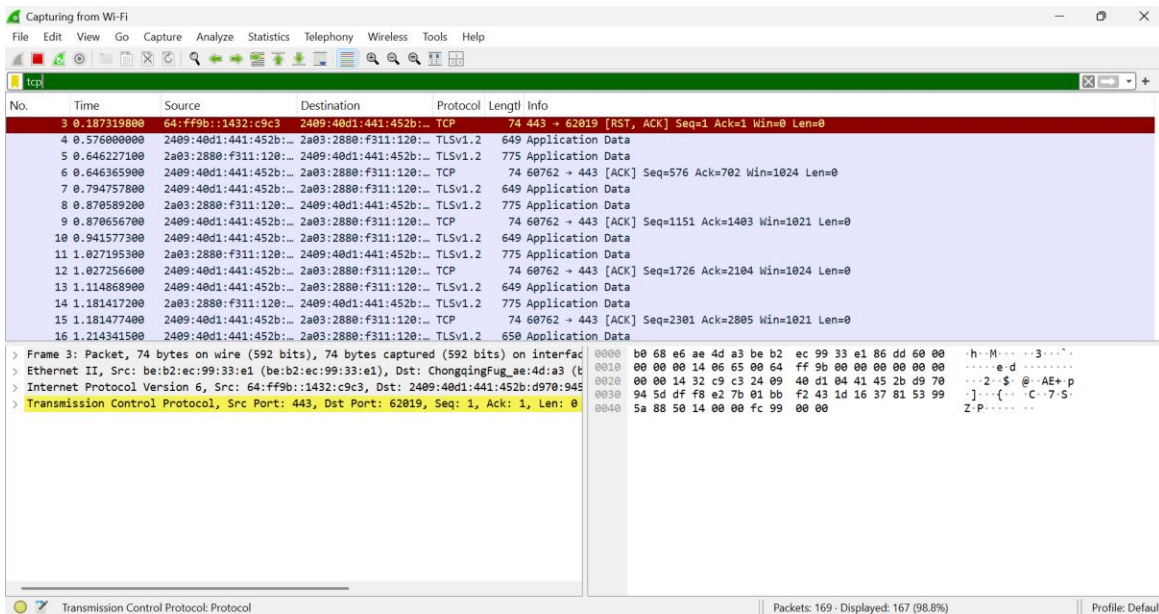
A network interface is how a computer connects to the network — common types are Ethernet (wired), Wi-Fi (wireless), and Loopback (local traffic only, used for testing). Picking the wrong interface is a common beginner mistake: capturing on Loopback shows only the machine's own local traffic, and capturing on the wrong Wi-Fi adapter means the traffic being looked for is simply never seen. For this experiment, the Wi-Fi interface was selected, as shown in the Wireshark title bar ("Capturing from Wi-Fi") across all three captures below.

EXPERIMENT 10

Part A — TCP Packet Capture (filter: tcp)

Procedure

1. Open Wireshark and select the active Wi-Fi interface.
2. Start the capture and let normal background traffic (an open website or app) generate some TCP activity.
3. Apply the display filter tcp.
4. Look for TLS Application Data records and any TCP control packets (SYN, ACK, RST) mixed in among them.
5. Select a control packet and expand the Ethernet, IP, and TCP layers in the detail pane to inspect its fields.



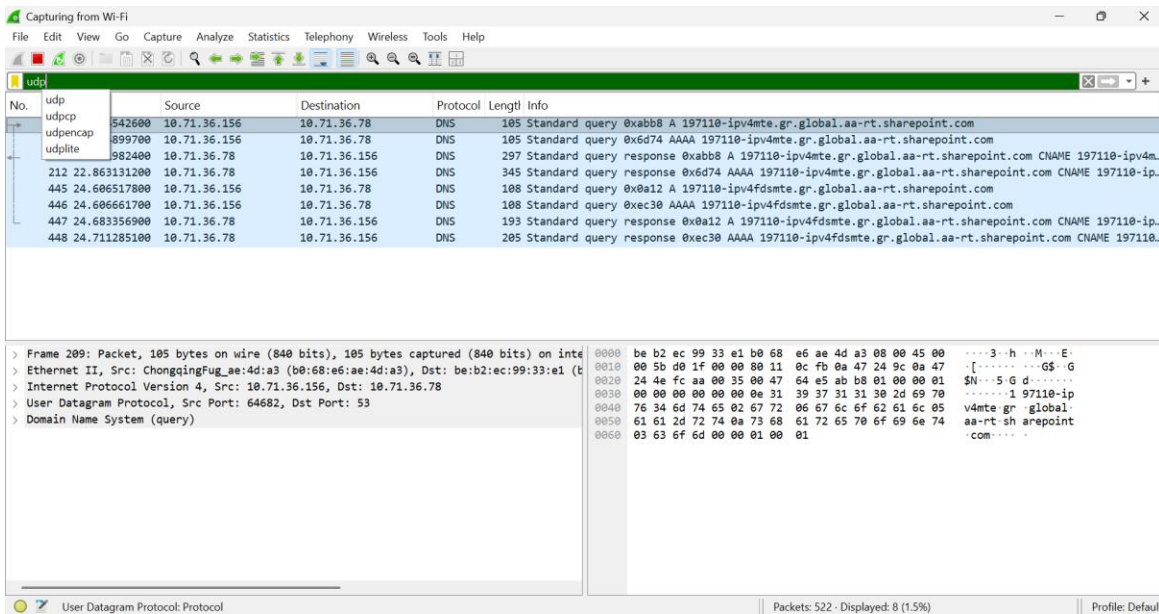
With the tcp filter applied, frame 3 (highlighted) shows a [RST, ACK] from port 443 to port 62019 — the remote server forcibly tearing down a stale connection (64:ff9b::1432:c9c3 is a NAT64-synthesised IPv6 address standing in for an IPv4 host). The detail pane below confirms this: an IPv6 header carrying the TCP segment, with Src Port 443, Dst Port 62019, Seq=1, Ack=1, Len=0. Underneath that, frames 4 onward show a separate, already-established TLSv1.2 session between the local address (2409:40d1:441:452b::...) and a remote host (2a03:2880:f311:120::...), alternating between Application Data records (649–775 bytes) and plain TCP ACKs (e.g. 60762 → 443 [ACK] Seq=576 Ack=702). Because TLS encrypts the payload, Wireshark can only report "Application Data" for these records — the actual HTTP content inside is not visible without the session keys.

EXPERIMENT 10

Part B — DNS Packet Capture over UDP (filter: udp)

Procedure

6. Keep Wireshark capturing on the same Wi-Fi interface.
7. Apply the display filter udp.
8. Trigger a name lookup (in this case, background traffic to a Microsoft 365 / SharePoint-hosted resource) and observe the DNS query/response pairs.
9. Select a query packet and expand the UDP and DNS layers to confirm the source/destination ports and query fields.



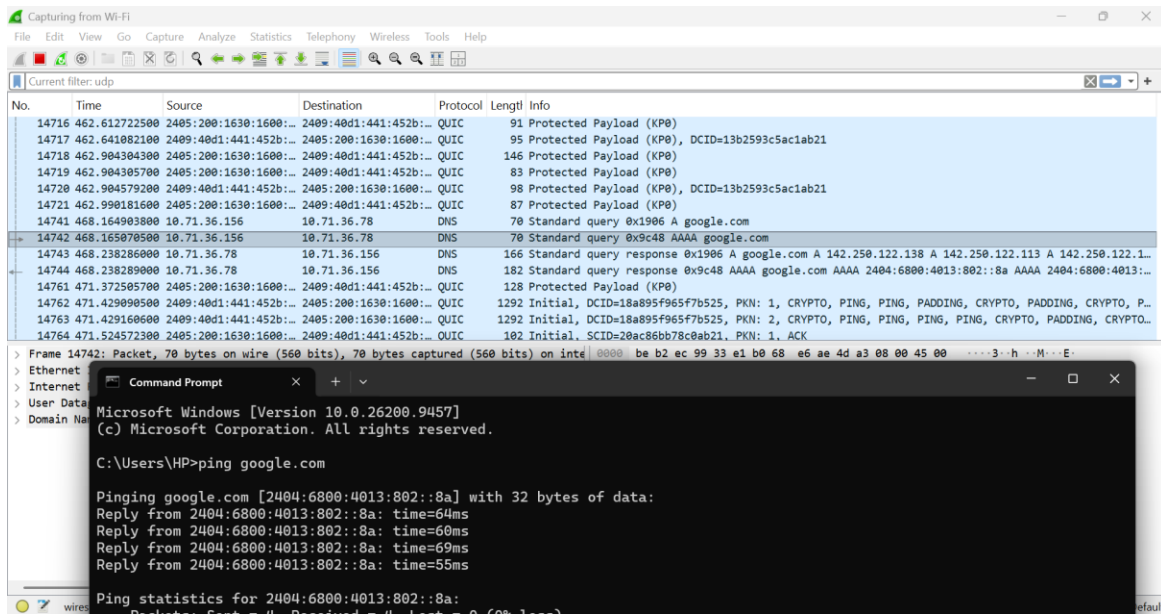
With the udp filter applied (the autocomplete dropdown shows udp was typed in), the capture shows two back-to-back DNS lookups from 10.71.36.156 to the local resolver at 10.71.36.78: an A query and an AAAA query for 197110-ipv4mte.gr.global.aa-rt.sharepoint.com, each answered with a CNAME redirect, followed by the same A/AAAA pattern for a related hostname, 197110-ipv4fdsmte.gr.global.aa-rt.sharepoint.com. This dual A + AAAA query pattern is typical of a dual-stack client resolving a name before connecting. The detail pane for frame 209 confirms the transport: Internet Protocol Version 4 (Src 10.71.36.156, Dst 10.71.36.78) carrying a User Datagram Protocol segment with Src Port 64682 and Dst Port 53 — the well-known DNS port — with a Domain Name System (query) sitting on top.

EXPERIMENT 10

Part C — QUIC Traffic & Pinging google.com (ICMPv6)

Procedure

10. Keep Wireshark capturing (QUIC also rides over UDP, so the same udp-style view applies).
11. Open Command Prompt and run ping google.com.
12. Let the ping command finish and note which IP version it resolved to and connected over.
13. Back in Wireshark, locate the DNS query/response pair for google.com and the surrounding QUIC packets.



Running ping google.com from Command Prompt (overlaid on the right) resolves google.com to the IPv6 address 2404:6800:4013:802::8a and reports four successful replies with round-trip times of 64 ms, 60 ms, 69 ms, and 55 ms — this is ICMPv6 Echo Request/Reply traffic, since Windows prefers IPv6 when a host publishes both an A and an AAAA record. In the packet list behind it, frames 14741–14742 show the two DNS queries that triggered this — 0x1906 A google.com and 0x9c48 AAAA google.com, both from 10.71.36.156 to 10.71.36.78 — and frames 14743–14744 carry the responses, listing several A records (142.250.122.138, 142.250.122.113, ...) and the AAAA record actually used for the ping (2404:6800:4013:802::8a, among others). The surrounding frames — labelled "Protected Payload (KP0)" and, further down, "Initial" packets carrying CRYPTO, PING, and PADDING frames — belong to an unrelated QUIC (HTTP/3) connection running at the same time; because QUIC encrypts almost everything once its handshake completes, Wireshark can identify the packet type and connection ID (DCID/SCID) but not the application data inside.

EXPERIMENT 10

Result

Wireshark was used to capture and analyse live traffic on the Wi-Fi interface across three filtered views: a tcp-filtered capture showing an in-progress TLSv1.2 session (steady-state Application Data and ACKs) alongside a TCP [RST, ACK] tearing down a stale connection; a udp-filtered capture showing paired A/AAAA DNS queries and CNAME-redirected responses for a SharePoint-hosted resource; and a third capture showing QUIC (HTTP/3) traffic running alongside a DNS lookup and ICMPv6 ping exchange for google.com, with round-trip times of 55–69 ms. Together these confirm how promiscuous-mode packet capture, via libpcap/Npcap, exposes the real, layered mix of protocols — TCP, TLS, DNS, UDP, QUIC, and ICMPv6 — that a single device generates during ordinary use.

Learning Outcome

After completing this experiment, the following learning outcomes were achieved:

- Understood how Wireshark uses promiscuous/monitor mode and libpcap/Npcap to capture packets directly from the OS kernel.
- Learned to select the correct network interface (Wi-Fi, in this case) before capturing.
- Captured and interpreted an in-progress TLSv1.2 session and distinguished Application Data records from plain TCP control packets (ACK, RST).
- Captured and interpreted paired DNS A/AAAA queries and their CNAME-redirected responses, and confirmed that DNS runs over UDP port 53.
- Observed QUIC (HTTP/3) traffic and understood why its payload appears only as "Protected Payload" once its handshake completes.
- Correlated a ping google.com command with its underlying DNS lookup and the resulting ICMPv6 Echo Request/Reply exchange.
- Practised using Wireshark display filters (tcp, udp) to isolate traffic of interest from a much larger full capture.